



Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en sistemas de Información
Programación I

Documentación Proyecto Final

Autor: Andy Antonio Ajin Rodríguez

Carnet: 9941-25-104

Catedrático: Ing. David Alvarez

Plan Fin de Semana

Sección "A"

31/05/2026

Proyecto Final: Sistema de Ventas e Inventario en C++

Contenido

Proyecto Final: Sistema de Ventas e Inventario en C++	2
1. Introducción	3
2. Estructura del Proyecto	3
3. Archivos y Funcionalidad	3
3.1 Productos	3
3.2 Ventas	4
3.3 Reportes	4
3.4 Ordenamiento	4
3.5 Menús	5
3.6 Main	5
4. Validaciones y Seguridad	5
5. Flujo General del Sistema	5
6. Conclusión	6

1. Introducción

Este proyecto implementa un sistema modular de ventas e inventario en C++, organizado en múltiples archivos .h y .cpp para mantener una estructura clara y profesional. El sistema permite:

- Registrar y gestionar productos.
- Registrar y mostrar ventas.
- Generar reportes y estadísticas.
- Ordenar productos por distintos criterios.
- Reiniciar el inventario con confirmación.

2. Estructura del Proyecto

El proyecto está dividido en módulos:

- Productos.h / Productos.cpp → Gestión de productos (CRUD).
- Ventas.h / Ventas.cpp → Registro y control de ventas.
- Reportes.h / Reportes.cpp → Generación de reportes y estadísticas.
- Ordenamiento.h / Ordenamiento.cpp → Funciones de ordenamiento y reinicio de inventario.
- Menus.h / Menus.cpp → Menús interactivos para el usuario.
- main.cpp → Punto de entrada del sistema.

3. Archivos y Funcionalidad

3.1 Productos

- Productos.h: Define la estructura Productos y prototipos de funciones.
- Productos.cpp: Implementa funciones como:
 - registrarproducto() → Añade un nuevo producto validando entradas.
 - mostrarproducto() → Lista todos los productos.
 - buscardatos() → Busca productos por ID o nombre.
 - modificardatos() → Permite modificar atributos de un producto.
 - cargarProductos() → Carga productos desde archivo binario.

Validaciones: uso de `cin.fail()`, `isalpha()` y excepciones con `try/catch`.

3.2 Ventas

- `Ventas.h`: Define la estructura `Ventas`.
- `Ventas.cpp`: Implementa funciones como:
 - `registrarVenta()` → Registra una venta, descuenta stock y guarda en archivo.
 - `mostrarventa()` → Muestra todas las ventas registradas.
 - `cargarVentas()` → Carga ventas desde archivo binario.

Uso de `localtime` y `strftime` para registrar fecha de venta.

3.3 Reportes

- `Reportes.h` / `Reportes.cpp`:
 - `reporteMenorStock()` → Lista productos con menor stock.
 - `reporteMasVendidos()` → Muestra productos más vendidos.
 - `reporteVentasDia()` → Ventas totales en una fecha específica.
 - `reporteVentasMesDia()` → Ventas por mes y día.
 - `mostrarEstadisticas()` → Estadísticas generales del inventario.
 - `exportarReporteTXT()` → Exporta reportes a un archivo `.txt`.

Validación de formato de fecha (YYYY-MM-DD).

3.4 Ordenamiento

- `Ordenamiento.h` / `Ordenamiento.cpp`:
 - `ordenarPorPrecio()` → Ordena productos por precio (asc/desc).
 - `ordenarPorStock()` → Ordena productos por stock.
 - `ordenarPorVentas()` → Ordena productos según ventas acumuladas.
 - `reiniciarInventario()` → Limpia vector y archivo `productos.dat` con confirmación.

Implementación con Selection Sort y uso de `strcmp` para comparar nombres.

3.5 Menús

- Menus.h / Menus.cpp:
 - gestionproductos() → Menú para productos.
 - gestionventas() → Menú para ventas.
 - gestionreportes() → Menú para reportes.
 - gestionorden() → Menú para ordenamiento.
 - menu() → Menú principal que conecta todo.

Cada menú usa try/catch para manejar errores y evitar caídas.

3.6 Main

- main.cpp:
 - Incluye todos los módulos.
 - Llama a menu() como punto de entrada.
 - Mantiene el código limpio y modular.

4. Validaciones y Seguridad

- Uso de cin.fail() para detectar entradas inválidas.
- Manejo de errores con try, throw, catch.
- Validación de cadenas con isalpha y control de tamaño con strncpy.
- Uso de nullptr en búsquedas para indicar punteros vacíos.

5. Flujo General del Sistema

1. Inicio → main.cpp llama a menu().
2. Menú principal → Usuario selecciona módulo.
3. Gestión de productos → CRUD de inventario.
4. Gestión de ventas → Registro y listado de ventas.
5. Reportes → Estadísticas y exportación.
6. Ordenamiento → Ordenar inventario o reiniciarlo.

7. Salir → Finaliza el programa.

6. Conclusión

Este proyecto demuestra:

- Uso de modularidad en C++ (separación en .h y .cpp).
- Aplicación de estructuras, vectores y archivos binarios.
- Implementación de validaciones robustas con excepciones.
- Organización clara mediante menús interactivos.

Es un sistema completo de ventas e inventario, adaptable a escenarios reales y con buenas prácticas de programación.